

Arhitectura Sistemelor de Calcul (ASC)
Examinarea finală
Varianta 1
(2024 - 2025)

Anul I, Semestrul I
7 februarie 2025
Cristian Rusu

Nume: _____

Prenume: _____

Grupa: _____

Completați aici totul cu majuscule.

Înainte de a începe, citiți cu atenție indicațiile următoare:

- Testul și rezolvarea sa vor fi disponibile online în zilele următoare.
- *Nu aveți voie cu laptop-uri sau alte dispozitive de comunicație.*
- *Nici calculatoarele de buzunar nu sunt permise.*
- *Vă rugăm să vă opriți telefoanele mobile.*
- Pentru întrebările cu răspunsuri multiple/simple folosiți tabelele puse la dispoziție.
- Acest test are 6 enunțuri totalizând 100 de puncte.
- Aveți la dispoziție 120 de minute pentru a completa examinarea.
- Mult succes!

Întrebarea 1. (22 puncte)

Completați tabelul de mai jos cu valorile numerice corecte. Toate numerele sunt naturale pe 16 biți. (Fiecare răspuns corect valorează 1 punct)

binar	octal	zecimal	hexazecimal
b0000110001000001			
	o04526		
		2419	
			0x05C7

a binar	a hexa	b binar	b hexa	a+b zecimal	a+b binar	a+b hexa
b0000001001		b0000111011				

a binar	a hexa	b binar	b hexa	axb zecimal	axb binar	axb hexa
	0x25		0x65			

Întrebarea 2. (13 puncte)

Completați tabelul de mai jos cu valorile numerice corecte. Toate numerele sunt întregi pe 12 biți. (Fiecare răspuns corect valorează 1 punct)

binar	octal	zecimal	hexazecimal
b1100011110			
	o1664		

a zecimal	a binar	a hexa	b zecimal	b binar	b hexa
	b1010110110		-24		

produs axb zecimal	produs axb binar	produs axb hexa

Întrebarea 3. (17 puncte)

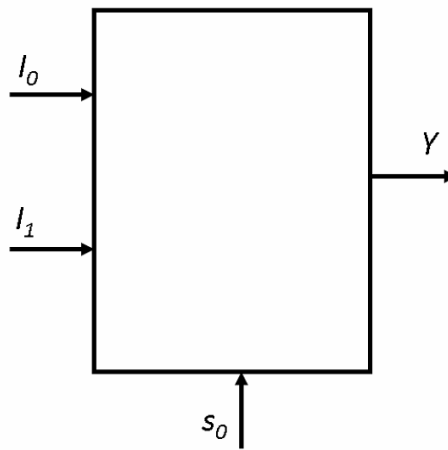
Răspundeți la următoarele întrebări scurte. Completați în tabelul de pe pagina anterioară.

- 3.1. Cine construiește primele sisteme de calcul care reprezintă numere în virgulă mobilă?
- 3.3. Matematician francez în onoarea căruia a fost numit un limbaj de programare în anii 80-90, este printre primii europeni care construiește un calculator mecanic.
- 3.5. În tabelul lui Mendeleev sunt 118 elemente. Fiecare element poate să aibă maxim 7 legături cu alți atomi. Câți biți are codificarea formulei H_2O (codificare cu dimensiune fixă).
- 3.7. Dacă un număr scris în binar are reprezentarea un șir de n biți de 1 atunci valoarea lui în zecimal este $2^n - 1$? Analog, care este valoarea în zecimal a unui număr care în hexazecimal este un șir de n cifre 1?
- 3.9. Avem două numere naturale a, b pe 2 biți (a este a_1a_0 și b este b_1b_0). Scrieți o expresie logică (folosind AND, OR, NOT, XOR) care verifică dacă a este egal cu b ?
- 3.11. Simplificați maxim expresia logică $(A \text{ AND } B) \text{ OR } (A \text{ XOR } B)$.
- 3.13. Avem un sistem de calcul pe care cineva a uitat să implementeze instrucțiunea de înmulțire pentru două numere naturale. Dar avem instrucțiunea de adunare pentru două numere naturale. Vrem să realizăm produsul $a \times b$ folosind doar instrucțiunea de adunare $a + b$. De câte ori este apelată această instrucțiune pentru a realiza înmulțirea? (vrem să apelăm de cât mai puține ori adunarea)
- 3.15. în formatul single float IEEE, care este reprezentarea hexazecimală a numărului -16?
- 3.17. în formatul single float IEEE, care este reprezentarea hexazecimală a numărului -3.875?
- 3.19. Fie A și B două numere în format IEEE 754 FP pe 32 de biți. Care este rolul secvenței următoare? $\text{NOT}(((A \gg 23) \text{ AND } 0x000000FF) \text{ XOR } (B \text{ AND } 0x7F800000) \gg 23)$
- 3.21. Pe un sistem cu arhitectura de calcul RISC-V avem următoarele două instrucțiuni: `add x5, x1, x2` și `sub x6, x5, x3`. Ce fel de hazard este prezent în această secvență?
- 3.23. Avem un sistem de calcul cu memorie ierarhică: memorie cache L1 de 1MB cu viteza de acces 1ns, memorie cache L2 de 5MB cu viteza de acces 5ns, memorie cache L3 de 10MB cu viteza de acces 10ns și memoria principală RAM de 32GB cu viteza de acces 50ns. Care este timpul minim de acces în memorie pe un astfel de sistem?
- 3.25. Într-o variabilă *suma* contorizăm câte numere impare sunt în vectorul x . Care este expresia logică/aritmetică fără *dacă* care calculează corect suma?
- 3.27. Acronimul SIMD înseamnă
- 3.29. Arhitectura de calcul care este recunoscută pentru performanța per Watt este
- 3.31. Codul sursă Java nu se compilează direct în cod assembly, ci în
- 3.33. GDDR este folosită în

Întrebarea 4. (12 puncte)

Am văzut în timpul cursului cât de important este un multiplexor (MUX). Răspundeți la următoarele întrebări și realizați următoarele cerințe:

- 4.1. (2 puncte) Desenați în schema de mai jos funcția logică pentru un MUX cu două intrări și o ieșire.
- 4.2. (4 puncte) Scrieți funcția logică pentru un MUX cu 4 intrări (I_0 , I_1 , I_2 și I_3) și două semnale de selecție (s_0 și s_1).
- 4.3. (6 puncte) În x86/x87 am văzut instrucțiunea *xchg* (care interschimbă două valori). Implementați această instrucțiune pentru două variabile digitale A și B . Desenați un circuit care are două ieșiri: fie A și B , fie B și A în funcție de un semnal de selecție s (dacă $s = 0$ atunci se schimbă ordinea, altfel nu). Folosiți cât mai mult MUX-uri. Pentru acest circuit scrieți și funcțiile logice.



Întrebarea 5. (16 puncte)

Aveți un sistem de calcul care suportă instrucțiunile Assembly x86 de bază (aritmice, logice, salturi). Din păcate, din cauza limitărilor sistemului acesta nu are implementate instrucțiuni de împărțire (adunare/scădere, înmulțire sunt implementate). Răspundeți la următoarele întrebări:

- 5.1. (4 puncte) Se dau două numere naturale $a > 1$, $b > 1$ și $a \geq b$. Scrieți pseudo-cod pentru o secvență aritmetică/logică care implementează operația de împărțire pentru a și b (avem nevoie de împărțirea întreagă) folosind repetat operația de adunare. De câte ori este utilizată operația de adunare?
- 5.2. (12 puncte) Acum, știm că $a \gg b$, a este mult mai mare decât b . Idem cu punctul 5.1., dar folosiți repetat și operația de înmulțire pentru a realiza împărțirea mai repede. De câte ori este utilizată operația de înmulțire?

Întrebarea 6. (20 puncte)

Avem o implementare a unei liste simplu înlănțuite într-un limbaj de programare. Elementele listei sunt numere întregi pe 4 octeți iar adresele în memorie sunt numere naturale tot pe 4 octeți. Implementarea folosește o funcție *adaugă_nod* pentru a adăuga câte un element în listă, la finalul acesteia. Această funcție alocă memorie HEAP (nu pe stivă) la fiecare apel (iar alocarea se face prin syscall la sistemul de operare). Presupunem că avem o situație în care avem nevoie să adăugăm în listă un număr mare de noduri N . Răspundeți la următoarele întrebări:

- 6.1. (4 puncte) Descrieți două probleme de performanță care apar la utilizarea funcției *adaugă_nod*.
- 6.2. (8 puncte) Propuneți soluții pentru problemele identificate la punctul 6.1. Hint: folosiți operații de blocuri de memorie, nu pe elemente individuale.
- 6.3. (2 puncte) Pentru soluțiile propuse la punctul 6.2. analizați eficiența de stocare (date versus adrese la noduri următoare).
- 6.4. (6 puncte) Cu soluțiile propuse anterior explicați cum accesați elementul $0 \leq i \leq N - 1$ din lista și cum maximizați utilizarea cache-ului. Presupunem că o linie de cache are 512 octeți.